COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	7/16/2026	**DATE SUBMITTED:**	7/21/2026
NAME:	Canoy Hail B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Table 2-1. Initial Driver Program

Code:

```
#include<iostream>
#include<string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std:: endl;
    }

    //deconstructor
    ~Student() {
        std::cout << "Destructor Called. " << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
```

```
28
29      //Display Attributes
30      void printDetails(){
31          std::cout << this->studentName << " " << this->studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      Student student1("Roman", 28);
37      Student student2(student1);
38      Student student3;
39      student3 = student2;
40
41
42      return 0;
43  }
```

Output:

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation:

- A student class is created and created with two private variables to store the name and the age, the variables are also set to private. The class is also set with constructor, destructor, copy constructor and the main function. If the constructor was given no values it will print the default values which are the John Doe as name and 18 as age. The destructor runs when the object is removed from the memory while the copy constructor creates a new object by copying another object. In the main function three students are created the student1, student2, student3. student 1 is named roman while having the age of 28 while student 2 copies the information from student one so basically student 2 is also named roman and is aged 28 and that's the copy constructor. while student 3 is created using the default values which is john doe, with the age 18. The output shows the constructor. destructor. copy constructor was called and it worked.

Table 2-2. Modified Driver Program with Student Lists

Code:

```cpp
#include<iostream>
#include<string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std:: endl;
    }

    //deconstructor
    ~Student() {
        std::cout << "Destructor Called. " << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
```

```
28
29      //Display Attributes
30      void printDetails(){
31          std::cout << this->studentName << " " << this->studentAge << std::endl;
32      }
33  };
34
35  int main() {
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
40      int ageList[j] = {15, 16, 18, 19, 16};
41
42      return 0;
43
44  }
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation:

- The code here is basically the same as the first one but the main function uses an array and names the students directly as carly, freddy, zack, code and ages 15, 16, 18, 19, 16. Only both constructor and destructor and both are called five times because there were 5 values of students names and age..

Table 2-3. Final Driver Program

Code:

```
#include<iostream>
#include<string.h>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std:: endl;
    }

    //deconstructor
    ~Student() {
        std::cout << "Destructor Called. " << std::endl;
    }

    //Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
```

```cpp

    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

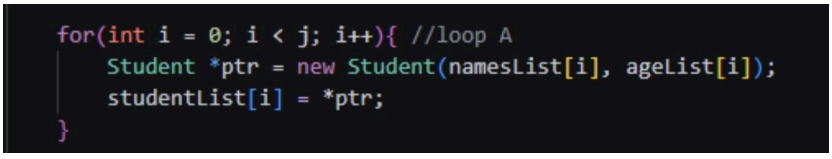
    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
    }

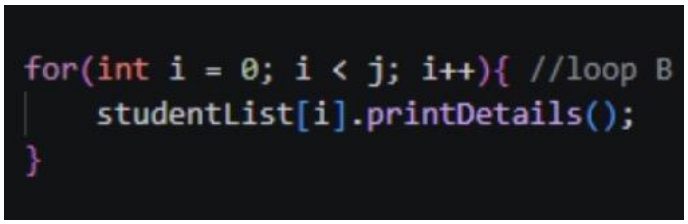
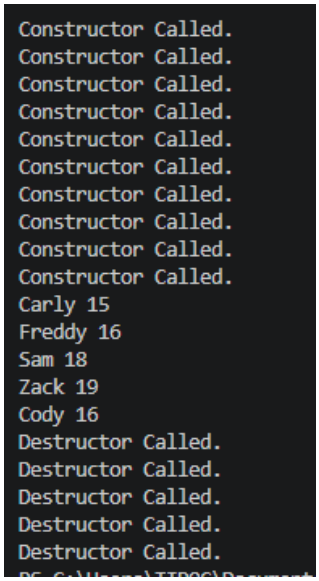
    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
    }

    return 0;
}
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Loop A	
Observation	- This loops creates new student array for each name and age. The constructor is then called whenever a new student is created and destructor is called whenever an object is deleted. This basically stores all the students value.

Loop B	
Observation	- This loop goes through the first loops array and accesses the student value and then the loop will print all the values of the students. Basically this loop is created to print all the stored information
Output	
Observation	- The constructor is called 10 times because the object or students are first created with the loop array; it created 10 because of the 5 ages and 5 names. The program then prints all the student values because of loop B. When the program ends all the values of the 5 students are destroyed that's why it printed 5 destructors.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

```cpp
#include <iostream>
#include <string>

class Fruit {
public:
    std::string name;
    int price, quantity;

    // Constructor
    Fruit(std::string n, int p, int q) {
        name = n;
        price = p;
        quantity = q;
    }

    // Copy Constructor
    Fruit(const Fruit &f) {
        name = f.name;
        price = f.price;
        quantity = f.quantity;
    }

    // Copy Assignment Operator
    Fruit& operator=(const Fruit &f) {
        name = f.name;
        price = f.price;
        quantity = f.quantity;
        return *this;
```

```
28          return *this;
29      }
30
31      // Destructor
32      ~Fruit() { }
33
34      int calculateSum() {
35          return price * quantity;
36      }
37  };
38
39  class Vegetable {
40  public:
41      std::string name;
42      int price, quantity;
43
44      // Constructor
45      Vegetable(std::string n, int p, int q) {
46          name = n;
47          price = p;
48          quantity = q;
49      }
50
51      // Copy Constructor
52      Vegetable(const Vegetable &v) {
```

```
53          name = v.name;
54          price = v.price;
55          quantity = v.quantity;
56      }
57
58      // Copy Assignment Operator
59      Vegetable& operator=(const Vegetable &v) {
60          name = v.name;
61          price = v.price;
62          quantity = v.quantity;
63          return *this;
64      }
65
66      // Destructor
67      ~Vegetable() { }
68
69      int calculateSum() {
70          return price * quantity;
71      }
72  };
```

```
73
74  int main() {
75
76      Fruit apple("Apple", 10, 7);
77      Fruit banana("Banana", 10, 8);
78
79      Vegetable broccoli("Broccoli", 60, 12);
80      Vegetable lettuce("Lettuce", 50, 10);
81
82      std::cout << "Apple Total: PHP " << apple.calculateSum() << std::endl;
83      std::cout << "Banana Total: PHP " << banana.calculateSum() << std::endl;
84      std::cout << "Broccoli Total: PHP " << broccoli.calculateSum() << std::endl;
85      std::cout << "Lettuce Total: PHP " << lettuce.calculateSum() << std::endl;
86
87      int total = apple.calculateSum() + banana.calculateSum()
88                  + broccoli.calculateSum() + lettuce.calculateSum();
89
90      std::cout << "\nGrand Total: PHP " << total << std::endl;
91
92      // Delete Lettuce
93      lettuce = Vegetable("", 0, 0);
94
95      std::cout << "\nLettuce deleted." << std::endl;
96
97      return 0;
98  }
99
```

```
Apple Total: PHP 70
Banana Total: PHP 80
Broccoli Total: PHP 720
Lettuce Total: PHP 500

Grand Total: PHP 1370

Lettuce deleted.
```

Observations:

- So in this program I created two classes which are for vegetables and fruit, to store their name, price and quantity. Then I created the constructor and copy constructor

to initialize the object's data when it's created while copy constructor is to copy assignment operator and allow the object to be copied correctly. And I declared a function calculatesunm() so that the total would be calculated with the formula quantity multiplied by price. And then I created the main function and created the objects I needed, which are the apple banana broccoli and lettuce, I created them with the format name, price and quantity. After that I replaced lettuce with an empty object to delete, then the destructor was called and destroyed the object.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For visualizing and knowing where or what to start and check errors.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I only used AI to get ideas and to check errors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.